

Parametrar till `main()`

Eftersom `main()` är en funktion borde den väl också kunna ta parametrar? Vilka är de i så fall, och var kommer de ifrån? Man kan faktiskt ge parametrar till `main()`, men de ges inte från någon anropande funktion, utan de kommer från de kommandoradparametrar som användaren ger. Denna funktionalitet gör det möjligt att ge parametrar till våra program. I de exempel vi hittills visat där `main()` förekommit har inga argument till programmen använts, och således heller inga parametrar till `main()`. Vill man kunna behandla även dessa bör man sätta till två parametrar till sin `main()`-deklaration, nämligen en `int` som kommer att innehålla antalet parametrar, och en *strängvektor* som innehåller de givna argumenten. Vi kommer att behandla vektorer i [Kapitel 9](#). Nedan kommer dock ett exempel som visar hur ett program kan se hur många parametrar det fått, samt skriva ut dessa:

```
#include <iostream>
#include <string>

int main (int argc, char * argv[]) {
    string Parameter;

    // hur många parametrar har vi fått?
    cout << "Vi har " << argc << " parametrar." << endl;

    // iterera över alla parametrar
    for ( int Index = 0; Index < argc; Index++ ) {
        // konvertera till en sträng och skriv ut
        Parameter = argv [ Index ];
        cout << "Parameter " << Index << " = " << Parameter << endl;
    }
}
```

Ovan ser du de två parametrarna `int argc` och `char * argv[]`. Dessa namn är de klassiska C-namnen på dessa parametrar, och det är bra att använda dessa så känns de lätt igen. Namnet `argc` kommer från *argument count*, och `argv` från *argument vector*. Den senare har datatypen `char *`, vilket är C:s sätt att representera strängar. Detta behandlas i [Appendix A](#). Hakparentesen efter `argv` indikerar att det är frågan om en *vektor*. Denna innehåller `argc` antal värden av typ `char *`. Vi konverterar dessa till C++-strängar för att de är lättare att hantera, samt skriver ut dessa. En körning av programmet ovan kunde t.ex. ge följande resultat:

```
% ./CommandLineArguments1 1 -2 3.001 foo bar "Hello world"
Vi har 7 parametrar.
Parameter 0 = ./CommandLineArguments1
Parameter 1 = 1
Parameter 2 = -2
Parameter 3 = 3.001
Parameter 4 = foo
Parameter 5 = bar
Parameter 6 = Hello world
%
```

Här ser vi att programmet fick sex parametrar förutom sitt eget filnamn. Intressant är närmast den sista parametern. Här har vi två ord i en enda parameter. Detta fungerar tack vare att orden är skrivna inom `" "`, varvid de tolkas som en enda parameter.

Om du kör programmet kommer du att märka att den första parametern alltid är programmets filnamn. Så även om du startar programmet helt utan parametrar så kommer alltid en att skickas till `main()`, nämligen namnet. Denna skickas alltid med så att programmet kan "veta vad det heter". Vi kunde modifiera ovanstående program en aning:

```
#include <iostream>
#include <string>

int main (int argc, char * argv[]) {
    string Parameter;
    string Namn;

    // namnet på programmet
    Namn = argv [0];

    // hur många parametrar har vi fått?
    cout << "Programmet '" << Namn << "' har " << argc - 1 << " parametrar." << endl;

    // iterera över alla parametrar förutom filnamnet
    for ( int Index = 1; Index < argc; Index++ ) {
        // konvertera till en sträng och skriv ut
        Parameter = argv [ Index ];
        cout << "Parameter " << Index << " = " << Parameter << endl;
    }
}
```

Vi vet alltid att `argv[0]` är programmets namn, och alla övriga värden är parametrar givna av användaren. Eftersom alltid minst en parameter skickas kan vi skriva kod som den ovan. Körning av programmet kunde t.ex. ge följande resultat:

```
% ./CommandLineArguments2 10 20 Hej
Programmet './CommandLineArguments2' har 3 parametrar.
Parameter 1 = 10
Parameter 2 = 20
Parameter 3 = Hej
%
```

Vi kan även enkelt göra ett program som kräver ett visst antal parametrar, t.ex. namn på datafiler:

```
#include <iostream>
#include <string>

int main (int argc, char * argv[]) {
    string Parameter;
    string Namn;

    // har vi fått tillräckligt med parametrar?
    if ( argc != 2 ) {
        // fel antal parametrar, skriv kort hjälptext och avsluta
        cout << "Fel antal parametrar!" << endl;
        cout << "Användning: " << argv[0] << " filnamn" << endl;
        return 1;
    }

    // korrekt antal parametrar eftersom vi kommit ända hit
    cout << "Antalet parametrar korrekt." << endl;
}
```

Programmet ovan kommer att kräva ett visst antal parametrar från användaren (1). Men eftersom det alltid finns programmets namn med som den första parametern måste vi jämföra med 2. Här ser vi

för första gången hur man kan returnera från `main()`. Vi har en `return 1` i `if`-satsens kropp. Denna kommer att avsluta programmet med returvärdet 1. Vart går då detta returvärde? Jo, det förmedlas tillbaka till kommandotolken, som kan använda detta värde för att kontrollera om programmet utfördes korrekt eller inte (se [avsnittet Returvärde från main\(\)](#)). Normalt låter vi kompilatorn returnera 0 om allt gick rätt till, men eftersom programmet ovan fick fel antal parametrar returnerar vi 1 istället för 0. Körning av programmet ovan kunde t.ex. ge:

```
% ./CommandLineArguments3
Fel antal parametrar!
Användning: ./CommandLineArguments3 filnamn
% ./CommandLineArguments3 foobar
Antalet parametrar korrekt.
%
```

[Förutgående](#)Funktionen `main()`[Hem](#)[Upp](#)[Nästa](#)

Omfattning (scoping)